

CHAPTER 01 : Boolean Retrieval

GPT LINK: <https://chatgpt.com/share/697f41f4-1284-800c-96ea-967a6c33f0a0>

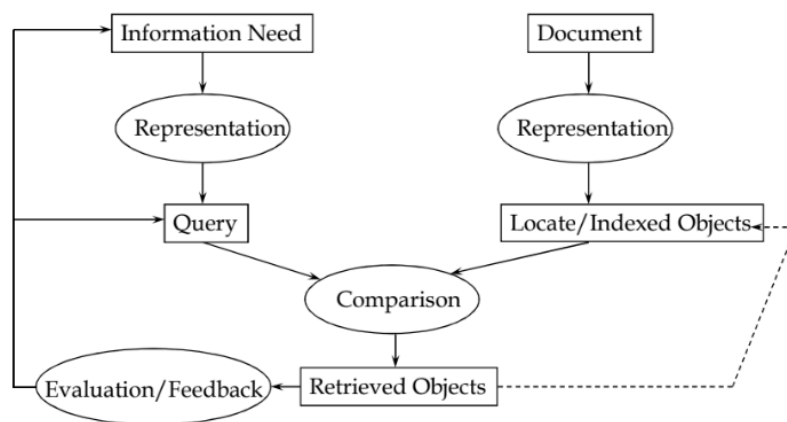
★ Model:

- An abstract representation of a process/object
- Used to study properties, draw conclusions, make predictions
- Quality of conclusions depends upon how closely the model represents the reality

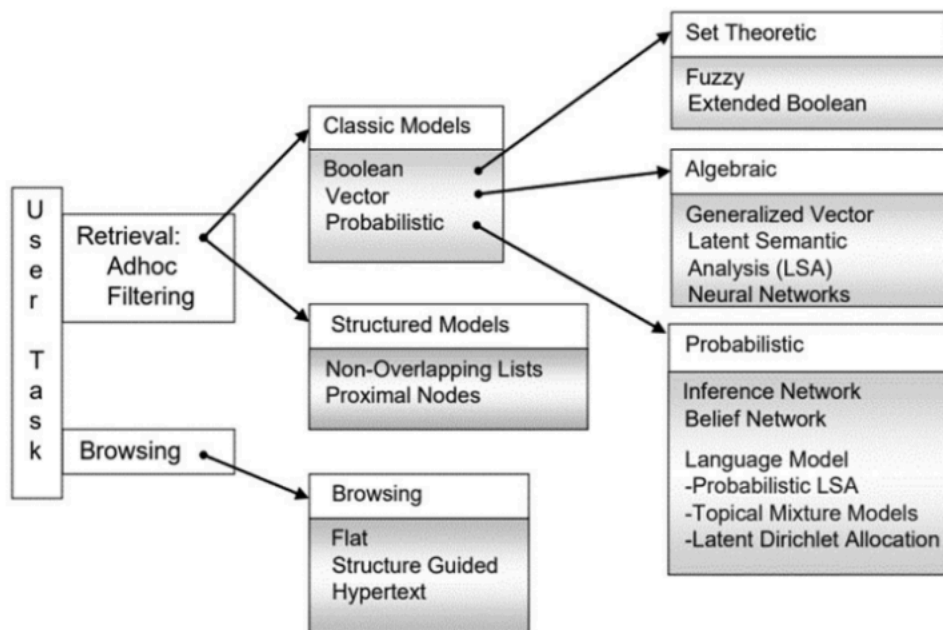
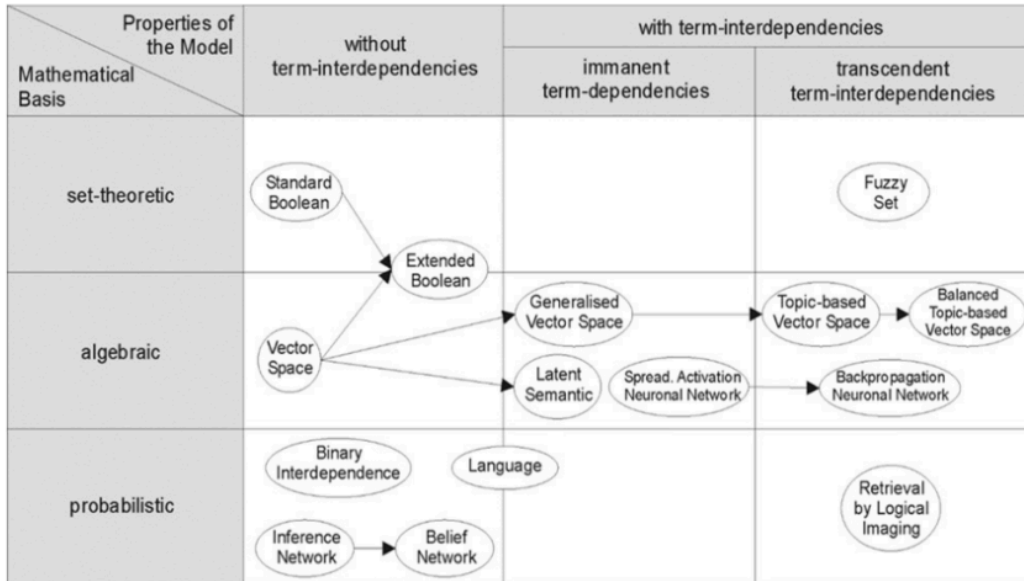
★ Retrieval Model:

- Describes the human and **computational processes** involved in **ad-hoc retrieval**
- These models define relevance, explicitly or implicitly

Basic Information Retrieval Process



IR Models



- The term “unstructured data” refers to data that does not have clear, semantically overt, easy-for-a-computer structure.

★ Basic IR Setting

- describes the **environment in which an IR system operates**.
- answers three questions:

Where is the data stored?

How big is the system?

What kind of data is being searched?

1 Offline vs Online IR

This dimension describes **when and how retrieval happens**.

◆ Offline IR

- Retrieval is performed on a **pre-collected, fixed corpus**
- Indexing is done **in advance**
- No real-time updates

Components

- **Corpus:** a static collection of documents
- Index is built once
- Queries are processed later

Examples

- Research paper datasets
- Archived legal judgments
- Offline digital libraries

📌 Advantages

- Faster querying
- Stable ranking
- Easier evaluation

Limitations

- No new data
- Cannot handle real-time updates

◆ Online IR

- Retrieval happens over a **dynamic, constantly changing collection**
- Documents are added, updated, or removed frequently

Components

- **Dynamic corpus** (e.g., WWW)
- Continuous crawling & re-indexing
- Real-time query handling

Examples

- Web search engines
- News portals
- Social media search

Challenges

- Freshness
- Duplicate content
- Index maintenance

Scale of Operation

This describes **how large the IR system is** and **who it serves**.

World Wide Web Level

- Massive, global-scale collections
- Billions of documents stored on millions of computers

- High diversity of topics and languages

Examples

- Web search engines
- Large knowledge portals

Key challenges:

- Ranking quality + Spam detection + Latency + Scalability
- not being fooled by site providers manipulating page content in an attempt to boost their search engine rankings

Personal / Individual Level

- Small-scale collections
- Usually limited to one user's data
- **issues** here include handling the broad range of document types on a typical personal computer,

Examples

- Desktop , email, cloud storage search

Key focus:

- Speed + Personal relevance + Privacy

Enterprise / institutional / Domain-Specific Level

- Medium to large collections
- Restricted to an organization or domain
- retrieval provided for collections such as a corporation's internal documents
- Docs stored on centralized file systems / one or a handful of dedicated machines provide search over the collection

Examples

- Company document repositories
- Legal databases
- Medical record systems

 Key focus:

- Accuracy + Access control
- Domain-specific vocabulary

★ An example information retrieval problem [Shakespeare]

⇒ Suppose you wanted to determine which plays of Shakespeare contain the words Brutus and Caesar and not Calpurnia

- **Human method:** read the book from start - end to determine if it will be in the result set
- simplest form of document retrieval is for a computer to do this sort of **linear scan** through documents. This process is commonly referred to as "**grepping**" through text, after the Unix command **grep**, which performs this process.
- **Grepping** → effective → allows wildcard pattern matching using regular expressions!
- Grepping is **not ideal** when we have to process large document collections quickly, or to when we want to allow flexible matching operations, and in ranked retrieval

Q. How do we avoid linearly scanning the text for each query?????

- By **INDEXING** the document in advance!!
- Read the file sequentially for Brutus AND Caesar and NOT Calpurnia, maintain a boolean status for each play.
- Check for play where : Brutus = 1 AND Caesar=1 and Calpurnia=0
- result is a **binary term-document incidence matrix**

★ Boolean Retrieval Model:

- Classical IR model
- Views each doc as just a **set of words (multi set)**
- Boolean retrieval model is a model for information retrieval in which we can **pose any query which is in the form of a Boolean expression of terms**, that is, in which **terms are combined with the operators AND, OR, and NOT**.
- Has no **ranking!!** - no concept of “how much” a document matches.
- Features either exist or dont exist in boolean model

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Antony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
...							

Figure 1.1 A term-document incidence matrix. Matrix element (t, d) is 1 if the play in column d contains the word in row t , and is 0 otherwise.

1 What is this table?

This is a **term-document incidence matrix**.

- Rows = terms (words)
- Columns = documents (plays)
- 1 = word appears in that document
- 0 = word does NOT appear

From left to right, the documents (columns) are:

1. Antony and Cleopatra
2. Julius Caesar
3. The Tempest
4. Hamlet
5. Othello
6. Macbeth

So each row is a **binary vector** for that word. So **every vector has 6 bits**, one per document, in this exact order

Vectors are **row based!!!**

- **Brutus: 110100**
- **Caesar: 110111**
- **Calpurnia: 010000**

4 Apply the query logic

Query:

Brutus AND Caesar AND NOT Calpurnia

Boolean logic rules:

- **AND** → bitwise AND
- **NOT** → bitwise complement (flip 0 ↔ 1)

- **NOT Calpurnia (flip bits): 101111**

Step 1: Brutus AND Caesar

markdown

110100

AND 110111

110100

Each 1 means "this document satisfies the query".

(Only positions where **both** are 1 stay 1)

Position	Document	Result
1	Antony and Cleopatra	✓
2	Julius Caesar	✗
3	The Tempest	✗
4	Hamlet	✓
5	Othello	✗
6	Macbeth	✗

Step 2: Result AND (NOT Calpurnia)

markdown

110100

AND 101111

100100

✓ Final Answer

The documents that contain Brutus AND Caesar BUT NOT Calpurnia are:

- Antony and Cleopatra
- Hamlet

- Bitwise operations make Boolean IR **fast and exact**

😞 advantages:

- Easy to implement
- Results r predictable, easy to explain
- Many diff features can be incorporated

⚠ problems!!!!

- Term doc matrix: sparse + fixed size
- EXACT MATCHING [operate,operation,operator etc wali problem] → **a word can have multiple meanings/context/variations**
- Complex query formulation
- All terms r equally weighted
- Flat results
- Info need is translated to boolean exp (query), which can be either v simple or way too complex. As a consequence, the Boolean model frequently returns either too few or too many documents in response
- NO RANKING!!!!!!!!!!
- No partial matching

★ Adhoc Retrieval Systems

- It is a system that aims to provide documents from within the collection that are relevant to an arbitrary user information need, communicated to the system by means of a one-off, user-initiated query.
- **Information need:** topic about which the user desires to know more
- **Query:** what the user conveys to the computer in an attempt to communicate the information need

★ Effectiveness of IR System

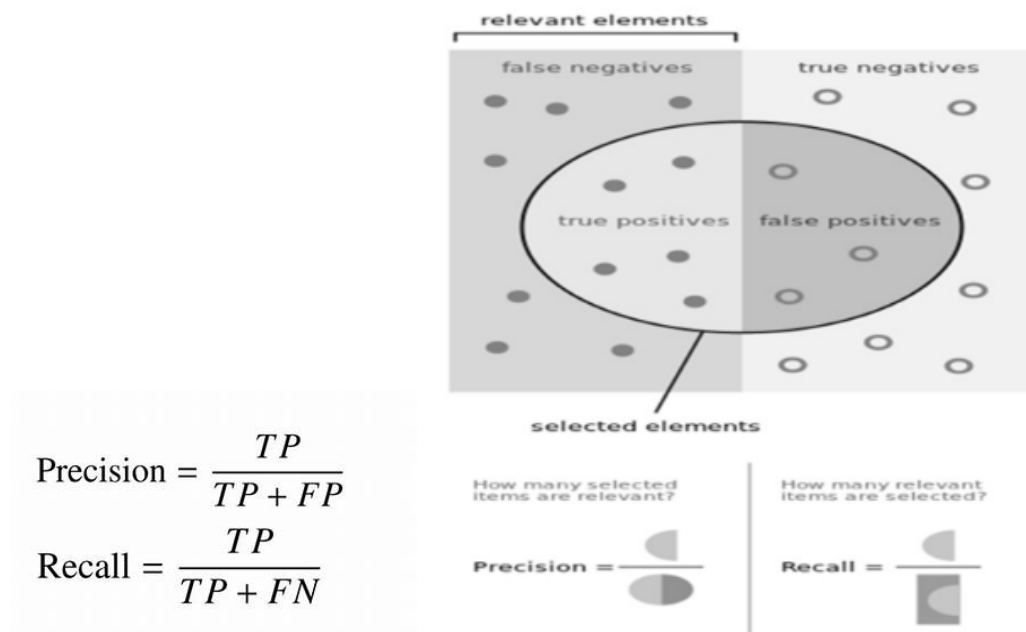
- quality of its search results.
- **2 factors**
 - 1) **Precision**
 - What fraction of the returned results are actually relevant to the information need
 - 2) **Recall**

- What fraction of the relevant docs in the collection were returned by the system

	Relevant	Irrelevant
Retrieved	Rel+Ret	Irrel+Ret
Non Retrieved	Non-Ret + Rel	Non-Ret + Irrel

Precision = relevant –retrieved / total retrieved

Recall = relevant –retrieved / total Relevant in collection



- An IR system returns 8 relevant documents, and 10 irrelevant documents. there are total of 20 relevant documents in the collection. what is the precision of the system, on this search, and what is its recall?
 - precision = $8/18 = 0.44$
 - recall $8/20 = 0.4$

[Week 1 b, Slide 9]

• $TP = 8$, $FP = 10$ → returned

• tot. relevant docs in collection :- 20

$$P = \frac{8}{20} \quad * \quad P = \frac{\text{relevant retrieved}}{\text{total retrieved}} = \frac{8}{8+10} = 0.44$$

$$* \quad R = \frac{\text{relevant retrieved}}{\text{tot. relevant in collection}} = \frac{8}{20} = 0.4$$

★ Points discussed in class:

1) jagged arrays:

- Dimensions not fixed
- Each row can have variable length [unequal sized rows]
- Good for sparse matrix

2) boolean model assumptions

- 1) User has proper knowledge of the terms that appear in doc
- 2) All docs r digitally processable (can get terms in them thru parsing)
- 3) Wtvr feature user hs presented, the doc containing those feature is relevant to user

[all these assumptions r very far frm reality but work for boolean model]

3) some terms

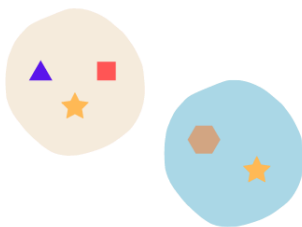
- $D = \{d_1, d_2, \dots, d_n\} \Rightarrow \text{docs}$ $n \gg \gg v \text{ large}$
- $Q = \{q_1, q_2, \dots\}$
- $D_1 \quad D_2$ $\rightarrow \text{sets}$

- $|D1| \cap |D2|$; $n1 > n2$
- $|D1 \cup D2| \Rightarrow \text{min: } n1, \text{ max: } n1 + n2$
- $\text{Rel}(d_i, q_j) = \{ \text{ranking order} \}$
- Term doc matrix is parsed $\rightarrow$ tokens r identified!

4 Short n Long Document Sets

SHORT DOC SET	LONG DOC SET
- docs r usually few words/sentences - search queries, tweets, news headlines, product titles etc - have very little context, therefore high ambiguity - low term freq - Semantic methods (embeddings) work better	- Can range from paragraphs to entire books - web pgs, research papers, legal docs, reports etc - rich in context, therefore easier to infer topic - TF-IDF works well - Document length normalization is important - low ambiguity

5 Jaccard similarity



$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

$$\frac{|\text{star}|}{|\text{triangle star square hexagon}|} = \frac{1}{4}$$

- one set is doc, one is query
- $A \cup B \rightarrow$ terms appearing twice r counted as 1
- No dense ranking, **flat results!**

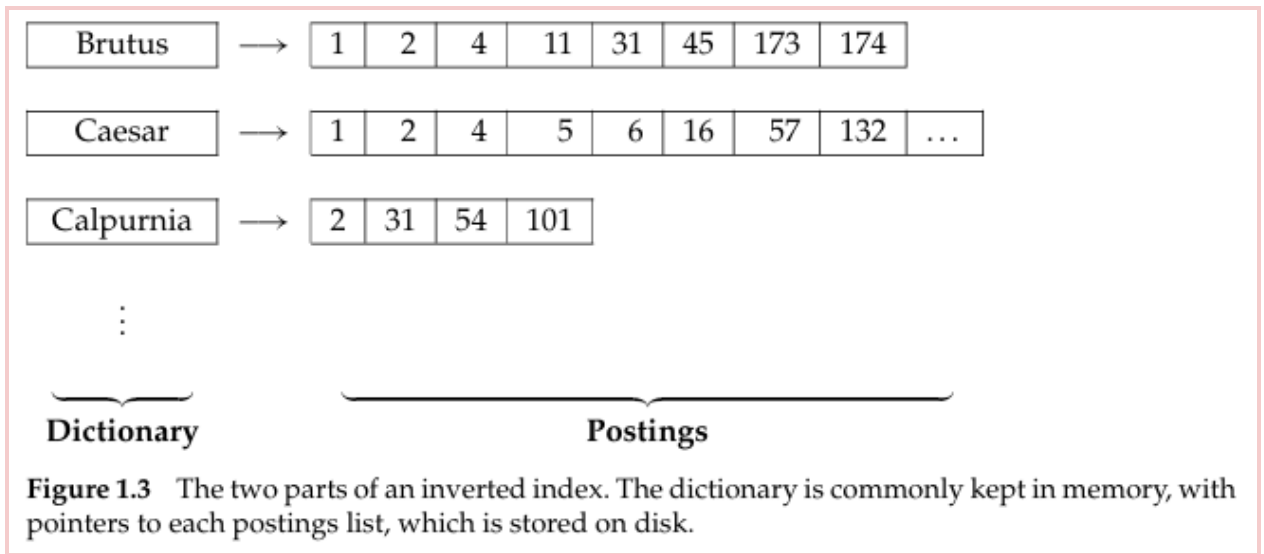
Jaccard Similarity

★ Inverted Index:

- an index of the vocabulary that occurs in a collection.
- Has 2 parts:
 - 1) **Lexicon/Dictionary/ Vocabulary** (terms, words, lexeme, tokens)

2) **Posting / posting-list/ inverted list** (document information, where the corresponding term belong)

- **Dictionary** kept in **memory**, pointing to each **posting lists** which is stored on **disk!**
- Dictionary stores the terms, and has a pointer to the postings list for each term
- more advanced indexing strategies such as **skip lists**, which require additional pointers.
- variable length arrays are more compact and faster to traverse.



- Dictionary is **scalable + extendable**. It can also have "extra variables" like; term freq, data type etc..
- Its sorted (a - z) which makes it easier to process → binary search applied (**log n**)
- Each document has a unique serial number → **document identifier (docID)**
- Posting list is a **singly linked list!!**
- **Dictionary terms point to posting lists!**

1. Collect the documents to be indexed:

Friends, Romans, countrymen. So let it be with Caesar ...

2. Tokenize the text, turning each document into a list of tokens:

Friends Romans countrymen So ...

3. Do linguistic preprocessing, producing a list of normalized tokens, which are the indexing terms:

friend roman countryman so ...

4. Index the documents that each term occurs in by creating an inverted index, consisting of a dictionary and postings.

Within a document collection, we assume that each document has a unique serial number, known as the document identifier (*docID*). During index construction, we can simply assign successive integers to each new document when it is first encountered. The input to indexing is a list of normalized tokens for each document, which we can equally think of as a list of pairs of term and docID, as in Figure 1.4. The core indexing step is *sorting* this list so that the terms are alphabetical, giving us the representation in the middle column of Figure 1.4. Multiple occurrences of the same term from the same document are then merged.⁵ Instances of the same term are then grouped, and the result is split into a *dictionary* and *postings*, as shown in the right column of Figure 1.4. Because a term generally occurs in a number of documents, this data organization already reduces the storage requirements of the index. The dictionary also records some statistics, such as the number of documents which contain each term (the *document frequency*, which is here also the length of each postings list). This information is not vital for a basic Boolean search engine, but it allows us to improve the efficiency of the search engine at query time, and it is a statistic later used in many ranked retrieval models. The postings are secondarily sorted by docID. This provides the basis for efficient query processing. This inverted index structure is essentially without rival as the most efficient structure for supporting ad hoc text search.

Doc 1

I did enact Julius Caesar: I was killed i' the Capitol; Brutus killed me.

term	docID	term	docID
I	1	ambitious	2
did	1	be	2
enact	1	brutus	1
julius	1	brutus	2
caesar	1	capitol	1
I	1	caesar	1
was	1	caesar	2
killed	1	caesar	2
i'	1	did	1
the	1	enact	1
capitol	1	hath	1
brutus	1	I	1
killed	1	I	1
me	1	i'	1
so	2	it	2
let	2	julius	1
it	2	killed	1
be	2	killed	1
with	2	let	2
caesar	2	me	1
the	2	noble	2
noble	2	so	2
brutus	2	the	1
hath	2	the	2
told	2	told	2
you	2	you	2
caesar	2	was	1

Doc 2

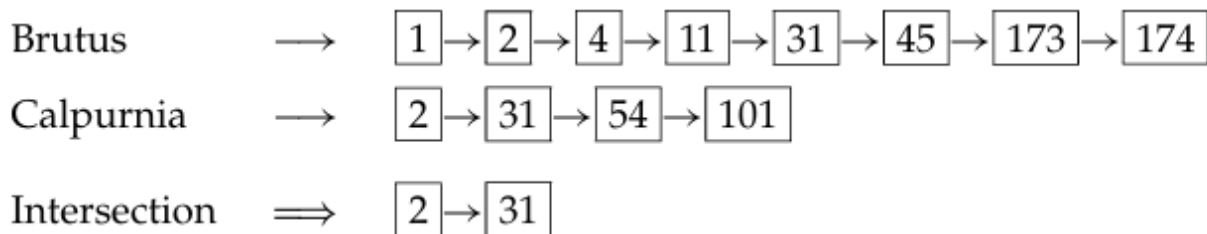
So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious:

term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
I	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

- Processing Boolean Queries of the form, for
- Example:
 - T1 OR T2
 - T1 AND T2
 - T2 AND T3 OR (T4 OR NOT T5)

example query : Brutus AND Calpurnia

- 1) Locate "Brutus" in the Dictionary
- 2) Retrieve its Postings
- 3) Locate "Calpurnia" in the Dictionary
- 4) Retrieve its Postings
- 5) Get the **intersection** of the two posting lists [**merging the posting lists**]



? **Exercise 1.1** [★] Draw the inverted index that would be built for the following document collection. (See Figure 1.3 for an example.)

Doc 1 new home sales top forecasts

Doc 2 home sales rise in july

Doc 3 increase in home sales in july

Doc 4 july new home sales rise

Exercise 1.2 [★] Consider these documents:

Doc 1 breakthrough drug for schizophrenia

Doc 2 new schizophrenia drug

Doc 3 new approach for treatment of schizophrenia

Doc 4 new hopes for schizophrenia patients

a. Draw the term-document incidence matrix for this document collection.

b. Draw the inverted index representation for this collection, as in Figure 1.3 (page 6).

Exercise 1.3 [★] For the document collection shown in Exercise 1.2, what are the returned results for these queries?

a. schizophrenia AND drug

b. for AND NOT (drug OR approach)

☀ intersecting posting lists:

- Maintain pointers into both lists and walk through the two posting lists simultaneously
- At each step, we compare the docID pointed to by both pointers
- If DOC IDs are same, we put that docID in the **results list**, and advance both pointers
- Otherwise we advance the pointer pointing to the smaller docID

```
INTERSECT( $p_1, p_2$ )
1   $answer \leftarrow \{ \}$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(answer, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7      else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8          then  $p_1 \leftarrow \text{next}(p_1)$ 
9          else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return  $answer$ 
```

Figure 1.6 Algorithm for the intersection of two postings lists p_1 and p_2 .

- lengths of the postings lists are x and y , the **intersection takes $O(x + y)$** operations
- The **complexity of querying is $\Theta(N)$** [$N \rightarrow$ no of docs in collection]
- can extend the intersection operation to process more complicated queries (with OR and NOT)

☀ Query optimization

- process of selecting how to organize the work of answering a query so that the least total amount of work needs to be done by the system
- The **order in which posting lists are accessed** matters!!!!

Q. What is the best order for query processing?

- Brutus AND Caesar AND Calpurnia

$$q=\{t_1, t_2, \dots, t_n\}$$

- For each of the t terms, we need to get its postings, then AND them together.
- The standard heuristic is to process terms in order of **increasing document frequency**: if we start by **intersecting the two smallest postings lists**, then all intermediate results must be no bigger than the smallest postings list, and we are therefore likely to do the least amount of total work.
- **LOW DF → HIGH DF** [DF → doc frequency]

Query

nginx

 Copy code

Brutus AND Caesar AND Calpurnia

Posting list sizes:

- Brutus → 300 docs
- Caesar → 500 docs
- Calpurnia → 20 docs

Efficient order

nginx

 Copy code

Calpurnia → Brutus → Caesar

- Start with 20 docs
- Intersect with 300 → maybe 5 left
- Intersect with 500 → maybe 2 left

👉 Only a few documents ever examined deeply.

```

INTERSECT( $\langle t_1, \dots, t_n \rangle$ )
1  terms  $\leftarrow$  SORTBYINCREASINGFREQUENCY( $\langle t_1, \dots, t_n \rangle$ )
2  result  $\leftarrow$  postings(first(terms))
3  terms  $\leftarrow$  rest(terms)
4  while terms  $\neq$  NIL and result  $\neq$  NIL
5  do result  $\leftarrow$  INTERSECT(result, postings(first(terms)))
6     terms  $\leftarrow$  rest(terms)
7  return result

```

Figure 1.7 Algorithm for conjunctive queries that returns the set of documents containing each term in the input list of terms.

⇒ **explanation of the above algorithm:**

- **[1]** Sort based on increasing DOC FREQ DF
- **[2]** Take the **posting list of the rarest term**, Initialize it as the current result
- At this point: *result* = all docs that contain the first (rarest) term
- **[3]** Remove the first term from the list
- **[4]** while: There are still terms left **AND** The result set is not empty
-  If *result* becomes empty:
 - No document can satisfy the full AND query
 - Stop early (optimization)
- **[5]** intersect: curr result set + posting list of next term
- **[6]** move to next term and repeat [4] - [6]

Q. how does this work with queries containing OR

 The query

(madding OR crowd) *AND* (ignoble OR strife) *AND* (killed OR slain)

This is a conjunctive query (AND) where each term is itself a disjunction (OR).

Key rule (conservative estimate)

For an OR expression:

CSS

A OR B

We estimate its size as:

$$DF(A \text{ OR } B) \approx DF(A) + DF(B)$$

📌 Why “conservative”?

- Because some documents may contain **both A and B**
- But adding frequencies guarantees we **don’t underestimate**
- Overestimation is safe for ordering

3 Example frequencies (hypothetical but realistic)

Let’s say the inverted index gives:

Term	DF
madding	5
crowd	100
ignoble	8
strife	60
killed	200
slain	20

4 Compute estimated size of each OR group

(madding OR crowd)

$$5 + 100 = 105$$

(ignoble OR strife)

$$8 + 60 = 68$$

(killed OR slain)

$$200 + 20 = 220$$

5 Order the OR groups by increasing estimated size

Smallest first ↓

1. (ignoble OR strife) → 68
2. (madding OR crowd) → 105
3. (killed OR slain) → 220

This is the optimal processing order.

★ extensions of boolean model:

- 1) Extended BM
- 2) Fuzzy Set BM

1 Extended Boolean Model

[The biggest issue is evident from the name that it is "boolean". This means that queries completely accept a document or reject it outright. This means some documents which might be a little relevant are also rejected. Extended Boolean solves this by enabling ranking with boolean queries]

The **Extended Boolean Model** improves the classical Boolean model by:

- Allowing **partial matching**
- Assigning **weights** to terms instead of just 0 or 1 [counts occurrences!]
- Producing a **relevance score**, enabling **ranking**

Key idea

Documents do not just *satisfy* or *fail* a query — they can match it **to different degrees**.

- Uses **soft AND / soft OR** operators
- Combines Boolean logic with **vector space concepts**

 Benefit:

- More flexible than strict Boolean IR
- Better user experience due to ranking

2 Fuzzy Set Model

- A document is **not just relevant or not relevant**
- It can be **partially relevant**.

So instead of:

- Relevant = 1, Not relevant = 0

we allow: **Relevance = any value between 0 and 1**

- Doc relevancy mapped to function

★ Ranked Retrieval (Vector Space Model, etc.)

- Users type free text queries → no operators, no structure
- system decides which documents best satisfy the query
- Eg: google search

★ proximity operator [used in proximity search]

[also an extension of the BM]

- a way of specifying that **two terms in a query must occur close** to each other in a document, where closeness may be **measured by limiting the allowed number of intervening words** or by **reference to a structural unit** such as a sentence or paragraph
- trade /s secret → same sentence
- trade /p secret → same paragraph
- Trade /3 secret → at most **3 words in between**
- **High precision** + **Less noise** + **More control** + Very useful in **legal, medical, technical texts**

Information need: Information on the legal theories involved in preventing the disclosure of trade secrets by employees formerly employed by a competing company.

Query: "trade secret" /s disclos! /s prevent /s employe!

Information need: Requirements for disabled people to be able to access a workplace.

Query: disab! /p access! /s work-site work-place (employment /3 place)

Information need: Cases about a host's responsibility for drunk guests.

Query: host! /p (responsib! liab!) /p (intoxicat! drunk!) /p guest

Note the long, precise queries and the use of proximity operators, both uncommon in web search. Submitted queries average about ten words in length. Unlike web search conventions, a space between words represents disjunction (the tightest binding operator), & is AND and /s, /p, and /k ask for matches in the same sentence, same paragraph or within *k* words respectively. Double quotes give a *phrase search* (consecutive words); see Section 2.4 (page 36). The exclamation mark (!) gives a *trailing wildcard query* (see Section 3.2, page 48); thus liab matches all words starting with liab. Additionally work-site matches any of *worksite*, *work-site* or *work site*; see Section 2.2.1. Typical expert queries are usually carefully defined and incrementally developed until they obtain what look to be good results to the user.

<Food for Thoughts>

1. Illustrate the difference between Text and Data Mining.

- **Text Mining** deals with extracting useful information and patterns from **unstructured or semi-structured text data** such as documents, emails, and web pages using NLP techniques.
 - **Data Mining** focuses on discovering patterns, trends, and relationships from **structured data** stored in databases using statistical and machine learning techniques.
-

2. Differentiate among Data, Information, Knowledge and Wisdom.

- **Data:** **Raw facts** and figures without context (e.g., numbers, symbols).
 - **Information:** **Processed data** that has meaning (e.g., average score of students).
 - **Knowledge:** **Interpreted information** combined with experience (e.g., understanding why scores are low).
 - **Wisdom:** **Ability to make** sound **decisions using knowledge** (e.g., improving teaching methods).
-

3. What do we mean by "Information Retrieval-IR"? Give an example of IR task. What are the different components of it?

- **Information Retrieval (IR)** is the process of finding relevant documents from a large collection in response to a user's query.
 - **Example:** Searching documents on Google using keywords.
 - **Components of IR:**
 - **Document collection** (corpus)
 - **Indexing system** (e.g., inverted index)
 - **Query processing**
 - **Matching and ranking mechanism**
 - **User interface**
-

4. What do we mean by Effectiveness of IR? How it is measured?

- The **effectiveness of IR** refers to **how well an IR system retrieves relevant documents** while avoiding irrelevant ones.
- It is measured using:
 - **Precision**
 - **Recall**
 - Sometimes F-measure or precision-recall curves

5. Why Precision and Recall both are required for understanding the effectiveness of an IR Systems?

- **Precision** measures how many retrieved documents are relevant, while **recall** measures how many relevant documents were retrieved.
- Both are required because:
 - High precision alone may miss many relevant documents
 - High recall alone may retrieve too many irrelevant documents

Together, they give a complete picture of IR effectiveness.

6. What is Adhoc information retrieval? Give an example.

- **Adhoc Information Retrieval** refers to retrieving documents relevant to a **one-time, user-initiated query** where the system has no prior knowledge of the user's intent.
 - **Example:** Searching "causes of climate change" on Google.
-

7. What are the best features of WESTLAW – Commercial Boolean Retrieval systems? What are some of its drawbacks?

Best features of WESTLAW:

- Precise Boolean queries using AND, OR, NOT
- **Proximity operators (/s, /p, /k)**
- **Phrase searching and wildcards**
- High control and transparency
- Suitable for legal and professional search

Drawbacks:

- Difficult for non-expert users
 - Requires carefully crafted queries
 - No natural relevance ranking
 - Low recall with strict AND queries
-

8. For a conjunctive query, is processing postings lists in order of size guaranteed to be optimal? Explain why it is, or give an example where it isn't.

- For **simple conjunctive (AND) queries**, processing posting lists in **increasing order of document frequency** is generally optimal because it minimizes intermediate results and comparisons.
- However, it is **not always guaranteed to be optimal** when:
 - **Query contains OR expressions**
 - **Term correlations** exist
 - Cost of accessing posting lists differs

In such cases, frequency alone may not give the best execution order.

9. What are the challenges for executing a query (Boolean expression) on a Boolean Model based IR system?

- Queries must be **precisely formulated**
 - **No ranking** of results
 - **Exact matching** only
 - Difficult to balance precision and recall
 - Large result sets for OR queries
 - Very small result sets for AND queries
-

10. Discuss pros and cons of using Boolean Model of IR for systems.

Pros:

- Simple and efficient
- Precise control over search
- Transparent behavior
- Suitable for professional domains

Cons:

- No ranking of documents
 - Requires expert users
 - Binary relevance (yes/no)
 - Poor handling of vague queries
-

11. How Extended Boolean Model addresses some of the shortcomings of BM for IR?

- The **Extended Boolean Model** improves the Boolean Model by:
 - Assigning **weights to terms** instead of binary values
 - Allowing **partial matching**
 - Supporting **soft AND and OR operators**
 - Enabling **ranked retrieval**

This reduces the rigidity of Boolean search while retaining user control.